

Team 04

Topic 10
CIT433027

Warm-Starting a Motion Planner Using Self-Supervised Learning

Technical University of Munich



Timo Matuszewski

timo.matuszewski@tum.de

Gabriel Salinas

gabriel.salinas@tum.de

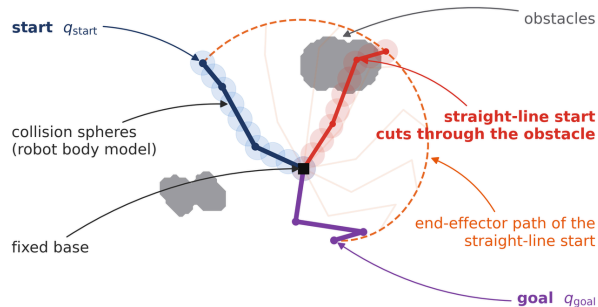
93.1% Crash-free warm-start trajectories
vs 61.8% cold-start

3.5x Fewer optimizer iterations
20.8 → 5.9 per solve

0 Labeled Trajectories Needed
Fully self-supervised

1 The Problem

Trajectory optimizers (CHOMP, TrajOpt, GPMP2) find smooth, collision-free paths, but they are **highly sensitive to their initialization**. A naive straight-line start almost always **cuts through obstacles**, so the optimizer wastes iterations or gets stuck in a colliding local minimum.



3-DOF planar arm in a 2-D workspace. The robot body is a chain of collision spheres; obstacles enter the network as a signed distance field (SDF).

2 Our Idea

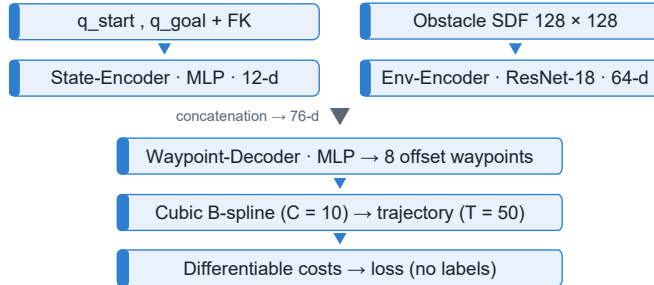
Learn the initialization. A neural network maps (start, goal, obstacle SDF) directly to a **near-feasible warm-start trajectory**, placing the optimizer next to the solution from the first step. It is trained self-supervised on differentiable planning costs: no demonstrations, no labeled trajectories.

Fast learning, reliable like an optimizer, and no labeled data.

3 Method

The network predicts the **initialization**, not the final path. A cubic B-spline over C control points defines the trajectory:

$$q(t) = \sum_{i=1}^C N_{i,3}(t) c_i, \quad t \in [0, 1]$$



$$c = c_{\text{line}} + \Delta c_{\theta}(q_{\text{start}}, q_{\text{goal}}, \text{SDF}), \quad \Delta c_{\theta}|_{\text{init}} = 0$$

The decoder predicts an offset from the straight-line baseline and is zero-initialized, so training starts exactly at the straight line.

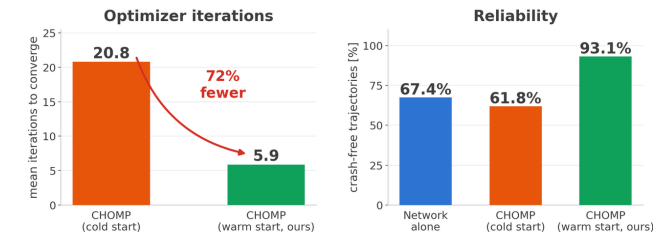
Self-supervised training signal

$$\mathcal{L}(\theta) = w_c \mathcal{L}_{\text{coll}} + w_j \mathcal{L}_{\text{lim}} + w_s \mathcal{L}_{\text{smooth}} + w_d \mathcal{L}_{\text{space}} + w_e \mathcal{L}_{\text{expl}}$$

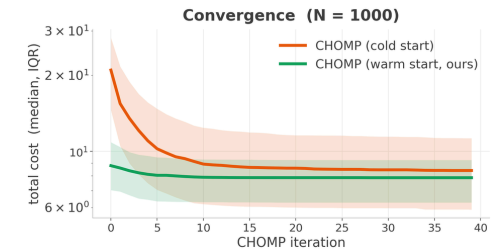
- Collision:** 3-zone penalty + continuous collision detection.
 - Joint limits:** quadratic violation penalty.
 - Smoothness:** squared joint velocity and acceleration.
 - Waypoint spacing:** discourages rushing through gaps.
 - Exploration:** escapes barely-outside local minima.
- PyTorch forward kinematics and bilinear SDF sampling make the pipeline end-to-end differentiable: no reference paths needed.

4 Results: Voxel Shapes (final model)

CHOMP warm-started with the network, N = 1000 held-out queries:



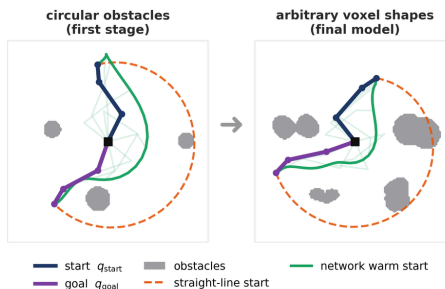
The warm start removes most of the optimizer work and is the only method above 90%. The network alone is the fastest.



Cold start (orange) begins far from feasible; the warm start (green) sits near the optimum from iteration 0.

Path quality is not traded away. Smoothness cost 1.07 → 0.88, joint-limit violations 291 → 90 per 1000 queries, and the crash-free rate stays consistent for 2, 3 and 4 obstacles.

5 Generalization: From Circles to Voxel Shapes



We first trained on **circular** obstacles, then extended the SDF pipeline to **arbitrary voxel shapes**, so the model is not limited to circles. The same pattern holds on both obstacle types:

Metric	Circular obstacles	Voxel shapes
Crash-free rate	71.9% → 97.6%	61.8% → 93.1%
Optimizer iterations	14.6 → 3.1	20.8 → 5.9
Planning time / query	145 → 35 ms	221 → 67 ms

Each cell: CHOMP cold start → warm start (ours).

6 Conclusion & Outlook

- A self-supervised warm start turns an unreliable optimizer into a **fast, reliable planner**, using **zero labeled trajectories**.
- Consistent across obstacle types: circles → arbitrary voxel shapes.
- Outlook: higher-DOF and 3-D task spaces, sim-to-real transfer.

Scan the QR to watch some simulations!



20.8 → 5.9 optimizer iterations

93.1% crash-free · 3.3x faster per query · 0 labels

Feasibility repaired by warm starting (network → CHOMP warm)

